Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O que o enunciado pede e o que você entrega. Não explique a implementação aqui.

O enunciado pede a construção de de um orquestrador de processos, capaz de trabalhar utilizando pipes, modos iterativos e workflow, com capacidade de funcionamento através de arquivos e montado em C.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Makefile	OK	Make run
Run (com pipe, sequential e parallel)	Feito	
Start, waitpid e jobs	Feito	
Modo batch	Feito	Adicionar no makefile, ao lado do ./ProcessFlow o nome do arquivo .txt
Registro de processos e mudança de pasta	Feito	

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make run
```

Executar (exemplo):

```
./ProcessFlow input.txt
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **tasks.c → Funções de manipulação da lista encadeada, compilação em modo pipe, sequential, paralllel e via background**
2. **terminal.c → Função de mudar de diretório**
3. **Back_processes.c -> funções para rastreamento de processos rodando em background (via lista encadeada**
4. **Task.h -> struct utilizada em tasks.c (para armazenar comandos cadastrados)**
5. **back_process.h->struct utilizada para cadastrar comandos rodando em background)**

Decisões técnicas:

1. **Uso de arquivos de funções separadas → Modularidade e melhor modificação atômica → Velocidade de execução e menor complexidade algorítmica**
2. **Busca de valor na lista(retornando ao início caso seja encontrado um dos valores)/compilação do comandos→ permite procurar de forma rápida e expandir de acordo com a quantidade→ Modular e eficiente, permitindo também expansão para mais comandos múltiplos**
3. **Abertura do arquivo e reciclagem das funções utilizadas no modo interativo -> permite manter funcionalidades de forma confiável e sequencial de acordo com o arquivo-> fácil de manter a segurança e polimorfismo geral, com desacoplamento e inversão de dependências com mínimo conhecimento**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Desenvolvimento modular e baseado no terminal	Partes são montados em pastas/módulos com funções que conversam entre si. A parte iterativa é a prioridade.	Por conta da estrutura, reunifiquei em monolito para garantir que módulos funcionam antes de separar em blocos de funções
S2	União em monolito e trabalho em 2 compiladores diferentes	A parte de conversão de string tornou-se um enorme problema, então resolvi simplificar unificando em monólito para resolver. Utilizei um compilador online junto do VsCode, para desenvolver versões rápidas de teste	Pouco prático para análise e para commits, então retornei, de forma parcial ao modelo anterior
S3	Separação em funções e compilador web GDB para desenvolver módulos críticos	Retornei ao modelo de divisão em funções, mas continuei a utilizar do compilador online Open GDB, em que adicionei uma versão monolítica do projeto, para elaborar módulos (considerados) de complexidade/risco de corrupção de memória [em caso de falha] críticos como workdir e pipe (também start, jobs,wait run parallel e run sequential)	Com o open GDB consigo debugar mais fácil (não sei como fazer no vscode) e trabalhar de forma dividida e comparativa, fazendo as correções adequadas e testes anti-erros

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	Criação do arquivos base e main S1	Seguindo a estratégia S1, comecei a montagem das bases mínimas para a construção adequada do sistema (funções de run e armazenamento)	A princípio houveram falhas de montagem, mas após resolver, funcionou adequadamente	As razões envolviam retornos indevidos (ou perdidos) das funções e a montagem básica do código	Do começo do código até (os primeiros 5 a 7 commits e, correspondentemente as primeiras linhas de evidências.log)	Linha 0 até linha 153 de evidências.log

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
2	Divisão das funções da Main (S1)	Após corrigir as bases, comecei a replanejar e dividir em funções, as funcionalidades da main (workdir, pipe, versões rudimentares de sequential, paralel e exit), assim como rascunhar-las. Também ocorreu o início do uso do OpenGDB como ambiente secundário e paralelo de desenvolvimento, para protótipo de Sequential e Parallel	O código tornou-se mais modular, entretanto, falhas de montagem nas funções e código incompleto afetam de forma significativa	As falhas que ocorreram, como as presentes entre as linhas 153 e 2200 de evidencias.log, ocorreram por código incompleto(como if's não fechados) / inoperante.	Após o 8º commit e após a linha 153 até 2200, tornou-se a linha predominante, até antes do incidente das duas branches No dia 18, pela primeira vez utilizei GDB, o que gerou a primeira versão (semi-parcialmente) funcional, e instável de sequential e parallel.	Após a linha 153, até a linha 2200 de evidencias.log, além dos commits de 8 até o commit anterior ao incidente, no caso o 2º commit do dia 20/8
3	Montagem de workdir S2 e um pouco antes da correção da branch, S3	Desenvolvi, depois de planejar as 3 funções citadas acima), por meio de compilador web, o que se provou ser bastante eficiente. Rascunhei as funções e fui atrás de seus módulos base para montar as chamadas POSIX da forma adequada. Antes (e também como consequência) do erro de branch, resolvi modularizar novamente, mas manter o uso de compilador online. Conclui o desenvolvimento de Workfile (função para mudar de	Durante o processo, acabei commitando (com o objetivo de desfazer) um commit acidental, sem intervalo entre os dois, gerando uma branch não intencional. Mas consegui corrigir através de 2 merges, e git pull para puxar a versão corrompida.	Ocorreu por causa de um commit para desfazer um commit acidental, mas que ambos ainda estavam sendo enviados. Logo, sem intervalo entre as duas ações foi criada a branch	Dia 20/8, no print , na mensagem enviada ao professor no slack entre as 9-10hrs da manhã, e nas linhas de evidências.log	Após a linha 2200 de evidencias.log e até a linha 2201(imediatamente após o incidente, evidencias, linha 2201, apenas foi retomado às 15hrs do dia 20/8, pós incidente das branches) Prints e commits do dia 20/8 captam o bug (no caso dos commits, a solução). E a mensagem de slack, também printada no horário próximo ao bug.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
		arquivo), antes do incidente de branches.				
4	Desenvolvimento dos pipes, parallel ,sequential, start, jobs,wait S3	Utilizei de pesquisas em stack overflow e no google para me auxiliar no desenvolvimento de Pipe, além de códigos de exemplo, do stack overflow e Geeks For Geeks, para engenharia reversa(nos casos do desenvolvimento de pipe e start) . Também foi aplicado, semi parcialmente, em parallel, sequential.	Foi possível desenvolver, de forma paralela correções e testes/debugging de forma mais eficiente que apenas utilizando o vscode (apesar das diferenças entre os dois ambientes)	As falhas que ocorreram estavam relacionadas a curva de aprendizado de pipes, processos em background e processos sequenciais e paralelos. Por vezes também, as diferenças entre os ambientes (VsCode e GDB) interferiram (o que gerou diversos commits corretivos, logo após implementar uma feature)	Iniciou-se no mesmo dia que ocorreu o incidente das branches, logo após o evento supracitado, encerrando-se no dia 22/08	Commits de número 25 até 40, e linhas 2201 a 342615 de evidencias.log, e print (a foto tirada no dia 22, extraída do stack overflow) de códigos escritos no open GDB.
5	Verificações de segurança, Modo Batch e README.md S3	Após a conclusão da tentativa 4, foquei em adicionar guard-rails e verificações de segurança para evitar falhas,além de, mantendo o mesmo modelo utilizado, desenvolver o modo Batch, t	Com as versões de segurança testadas no OpenGDB, aplicar-las no Vscode tornou-se algo bastante seguro e eficiente		Dia 23/8, com os commits executados no dia	Commits 41 a 43 no github

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

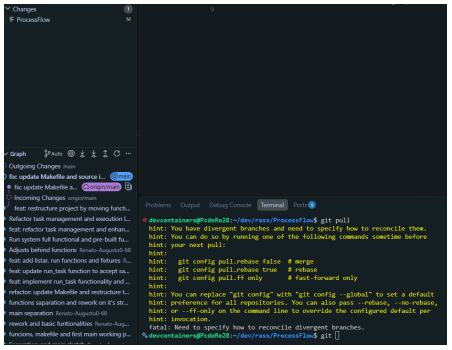
```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

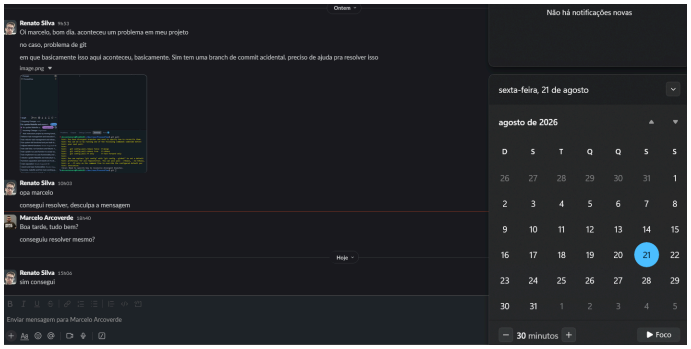
- Erro principal: o momento do bug mais relevante da semana.
- Validação final: teste passando / saída correta.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

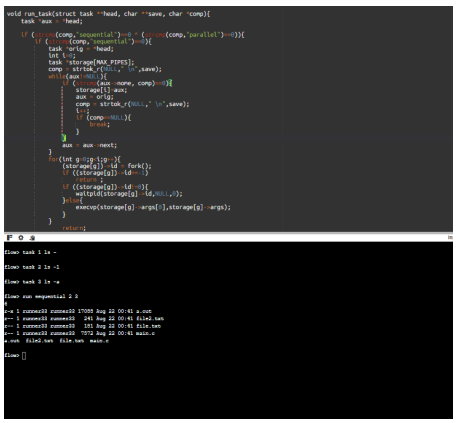


Branch acidentalmente criada

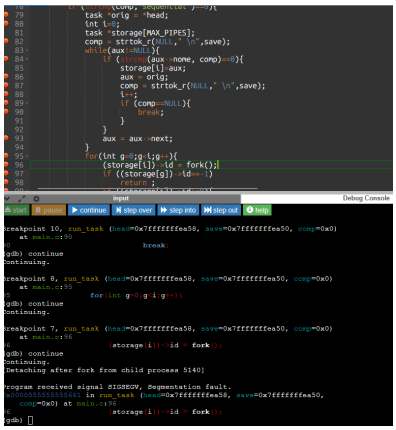
E pedido de ajuda ao professor



Demorei um bom tempo para resolver, mas bastou refazer o merge e re-comittar, além de usar commandos git como git pull (duas vezes), para recuperar a versão pré-corrupção e continuar dalí. Evidencias.log , não captou isto



Uma das primeiras versões(dia 18) (PARCIALMENTE, pois era insegura) funcionais de run sequential (era instável por não possuir verificação de valor na lista encadeada. Entretanto, foi pioneira no modelo arquitetural que usei em run pipe, sequential, parallel e start (envolvendo buscar o valor dentro da lista, voltar ao inicio da lista se encontrar e continuar até encontrar. E, fora do loop de busca, executar a ação solicitada, que é armazenada em um array auxiliar)



Correlata a imagem acima (trata-se da mesma versão, com mínimas diferenças), mostra a razão de ser um modelo instável e que gerava memory leaks



Tirada dia 22, essa imagem mostra um dos códigos oriundos do StackOverflow, que fiz engenharia reversa para desenvolver start e wait

7. Uso de IA
Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas): A IA foi utilizada de forma pontual, e em momentos específicos, como em explicar sintaxe de funções desconhecidas (como fork e strtok_r) assim como em explicações de seu comportamento.

Prompts principais (liste, sem história): Todos os prompts principais,quando utilizados seguiram o seguinte modelo: “me explique x, sem gerar código, de forma detalhada e concisa” ou(no caso de busca de sintaxe): “me explique, sem gerar código, como funciona a sintaxe de x” . Algumas variantes com palavras diferentes também ocorreram, mas os modelos/objetivos dos prompts se mantiveram sempre os mesmos. (Não foi possível recuperar os prompts utilizados, pois o histórico foi perdido)

O que validei manualmente e como: Avaliei e validei, de forma manual, cada etapa de desenvolvimento, de sua primeira e não funcional versão até a última, através de testes simples com printf e debugging, até mesmo em compilador web, rodando versões modificadas e/ou anteriores a mudanças, assim como através de engenharia reversa em códigos de sites como Geeks for Geeks e Stack Overflow, geralmente compilados também no compilador web.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

Eu melhoraria, caso possível, o sistema de verificação para inputs inválidos, assim como não definir o planejamento do sistema no primeiro momento e o incidente das branches. Também, se pudesse, eu

utilizaria uma arquitetura (a utilizada em start, run pipe, run parallel e run sequential), menos vulnerável a falhas de sincronização de fork-wait, e também a falhas de inputs do usuário. Se possível, também desenvolveria, de forma mais confiável a parcela responsável por run parallel, pois, por vezes, determinados comandos (como nano), acabam gerando comportamentos imprevisíveis quando rodados em paralelo.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Sim
Preenchi a seção de uso de IA	SIm
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico.

Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

Se eu tivesse mais 2 horas, teria desenvolvido uma função de verificação geral, em busca de valores nulos/ não encontrados dentro das listas encadeadas que utilizei. Assim como melhorar algumas nomenclaturas de funções e isolar dependências (como dependências de comportamentos da função fork e execvp). Por fim, caso tivesse mais tempo, trataria a ordem de processos rodando em paralelo e de como expor os valores de processos rodando em background, o que não consegui resolver adequadamente (no caso de processos em background).

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.